

Final Phase: Complete Documentation & Project Packaging

To wrap up your production deployment experiment, we will prepare your project files for an external reviewer. This ensures that any team member can clone, launch, and test your entire engine infrastructure in under **60 seconds**.

1. Finalize Project Documentation Layout

Open your project's root `README.md` file and replace its contents with this clean, production-grade layout context.

markdown

```
# Full-Stack Persistent Task API

A lightweight, robust Task Tracking API backed by a persistent PostgreSQL database instar

## 🚀 One-Command Quick Start

Run the following commands in your shell to clone, configure, and boot the entire live ap

```bash
1. Clone the project and step into the directory
git clone <YOUR_GITHUB_REPOSITORY_URL>
cd <YOUR_PROJECT_FOLDER>

2. Copy the environment variables template configuration
cp .env.example .env

3. Boot up the entire unified server stack
docker compose up --build
```

---

## ⚙️ Configuration & Variables

All sensitive parameters are decoupled from code logic using environment layers. The vari

Variable Keys	Placeholder Sample Value	Purpose Description
```

```
| `DATABASE_URL` | `postgres://postgres:dev@db:5432/tasks` | Network routing string used
```

📖 API Endpoint Routing Reference Table

| Route | Pattern | Method | Expected Body / Params | Response Codes | Description |
|--------------|----------|----------------------------------|------------------------------|----------------------------|-------------|
| :--- | :--- | :--- | :--- | :--- | |
| `/tasks` | `GET` | `?done=true`, `?search=term` | `200 OK` | Retrieves tasks filtered c | |
| `/tasks/:id` | `GET` | URL Parameter: `id` | `200 OK`, `404 Not Found` | Fetches a sing | |
| `/tasks` | `POST` | `{"title": "Task name"}` | `201 Created`, `400 Bad Req` | Creates a | |
| `/tasks/:id` | `PUT` | `{"title": "New", "done": true}` | `200 OK`, `400 / 404` | Safel | |
| `/tasks/:id` | `DELETE` | URL Parameter: `id` | `204 No Content`, `404` | Completely de | |

📋 Production Verification Logs

Live API Call Sample (`curl -i``)

```
```http
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 187
Date: Mon, 24 Aug 2026 15:45:00 GMT
Connection: keep-alive
```

```
[
 {"id":1,"title":"Buy groceries","done":false},
 {"id":2,"title":"Clean the room","done":true},
 {"id":3,"title":"Study API design","done":false}
]
```
```

Database Internal Storage Verification

*(Be sure to insert/embed your verification screenshot here! Example command to generate

```
```text
```

id	title	done
-----+-----+-----		
1	Buy groceries	f
2	Clean the room	t
3	Study API design	f

(3 rows)

```
```
```

Use code with caution.

2. Guard Against Leaks (Security Verification Checklist)

Before executing your deployment pushes to the cloud, run these terminal auditing checks to protect against credential compromises:

1. Verify Ignored Configuration Files:

bash

git status --ignored

Use code with caution.

Make sure your active live `.env` file with real development secrets appears under your ignored parameters list, not your staged file stack.

2. Verify Tracked Configuration Blueprints:

bash

git status

Use code with caution.

Confirm that `compose.yaml`, `Dockerfile`, `db.js` (or `database.py`), `server.js` (or `main.py`), and `.env.example` are properly tracked.

3. Git Save, Commit, and Publish Log Progress

Execute the following sequence in your root working shell terminal to push your infrastructure modifications online to your remote Git platform:

bash

1. Stage all your updated engine structures and project documents

```
git add server.js main.py db.js database.py Dockerfile compose.yaml .env.example README.md
```

2. Pack everything cleanly under the required structural commit signature

```
git commit -m "Stage 5: one-command stack + docs"
```

3. Ship your development cycle tree records straight to GitHub

```
git push origin main
```

